SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks:50**

Annexure A

DATA INTERPRETATION

Code of Conduct:

I K.Sabitha (192524308) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
K.Sabitha

Annexure A

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given Three Address Code

t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p

Analysis

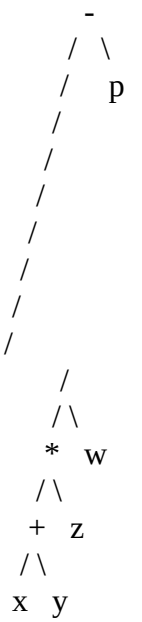
The operations are performed sequentially:

1. t1 stores x + y.
2. t2 multiplies the result by z.
3. t3 divides the result by w.
4. t4 subtracts p.

Therefore, the equivalent expression is:

t4 = ((x + y) * z / w) - p

Expression Tree



Conclusion

Temporary variables t1, t2, t3, t4 break a complex expression into simple operations. They make code generation easier and allow the compiler to reuse intermediate results, reducing unnecessary computations.

Q2. Symbol Table and Memory Allocation

Given Data

Variable	Data Type	Size
p	char	1 byte
q	double	8 bytes
r	int	4 bytes
s	float	4 bytes

Memory Calculation

Total = 1 + 8 + 4 + 4

= 17 bytes

Memory Organization

Assuming normal alignment, q may require an address divisible by 8. Therefore, padding may be inserted before q.

Example:

Variable	Size	Possible Memory
p	1 byte	1 byte
Padding	7 bytes	Alignment
q	8 bytes	8 bytes
r	4 bytes	4 bytes
s	4 bytes	4 bytes

Total including padding = 24 bytes.

Conclusion

Data types determine the amount of memory required. Alignment may add padding so that variables such as double are stored at suitable memory addresses. Thus, although the actual data size is **17 bytes**, aligned memory may require **24 bytes**.

Q3. Optimization in Intermediate Code

Given Code

t1 = m * n

t2 = p + q

t3 = m * n

t4 = t2 - t3

Execution Analysis

- m * n is calculated and stored in t1.
- p + q is calculated and stored in t2.
- m * n is calculated again for t3.
- t2 - t3 produces the final result.

The expression m * n is calculated **twice**, which is unnecessary.

Optimized Code

t1 = m * n

t2 = p + q

t4 = t2 - t1

Comparison

Code	**Multiplication Operations**
Original	2
Optimized	1

Conclusion

This is an example of **Common Subexpression Elimination**. Reusing t1 removes redundant computation, reduces execution time and improves CPU efficiency.

Q4. Control Flow and Boolean Expression Handling

Given Expression

if (x > y || p < q)

Intermediate Code

if x > y goto L1

if p < q goto L1

goto L2

L1: ...

Control Flow

Condition	Action
x > y is TRUE	Jump directly to L1
x > y is FALSE	Check p < q
p < q is TRUE	Jump to L1
Both FALSE	Jump to L2

Analysis

The OR (||) operator uses **short-circuit evaluation**. If x > y is already true, there is no need to evaluate p < q.

Conclusion

Short-circuit evaluation reduces unnecessary condition checking and improves execution efficiency, especially when Boolean expressions contain multiple conditions.

Q5. Backpatching and Flow Control

Given Code

Instruction	Target
200: if x == y goto _	Unknown
201: goto _	Unknown

Given:

truelist = {200}

falselist = {201}

Backpatching

Suppose the true condition should execute instruction **205** and the false condition should execute instruction **210**.

After backpatching:

Instruction	Updated Target
200: if x == y goto 205	205
201: goto 210	210

Analysis

- truelist contains jumps that must go to the **true branch**.
- falselist contains jumps that must go to the **false branch**.
- Backpatching fills the target addresses when they become known.

Conclusion

Backpatching is important in intermediate code generation because jump targets may not be known immediately. It ensures correct control flow for Boolean expressions and conditional statements while keeping code generation flexible and efficient.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand